REPORTE PARCIAL 2

Proyecto: Desarrollo del juego UNO

INTEGRANTES DEL EQUIPO

Selyan Nieto Aguirre

Pedro Axel Llarena Padilla

Nahima Suzette Rivera Hernández

Karla Andrea Durán Segura

1. Descripción general del proyecto

El proyecto consiste en el desarrollo de una versión digital del juego de cartas UNO mediante C# y Windows Forms. El sistema integra la lógica completa de la partida, la representación visual de las cartas, una interfaz gráfica para cuatro jugadores y el almacenamiento de información de las partidas mediante una base de datos MySQL comunicada con el programa a través de una API.

La aplicación contempla reglas como reparto de cartas, validación de jugadas, cambio de turnos y dirección, cartas especiales, selección de color, declaración de UNO, desafío del +4, detección del ganador y conteo de puntos. El desarrollo se organizó mediante Git y GitHub, utilizando ramas independientes para distribuir el trabajo y posteriormente integrar los cambios a la rama principal.

El diseño visual utiliza una paleta de colores pastel y recursos gráficos personalizados para mantener una interfaz coherente, clara y agradable. El proyecto se divide en componentes que pudieron desarrollarse de forma paralela: motor del juego, recursos visuales, interfaz gráfica, base de datos/API y documentación técnica.

2. División de tareas entre los integrantes del equipo

El trabajo se repartió de acuerdo con los componentes principales del proyecto. A continuación se presenta la distribución final de responsabilidades considerada para este reporte.

Selyan Nieto Aguirre
Interfaz gráfica principal, Windows Forms y Carta.cs

- Programación de la clase Carta.cs para representar el color, tipo y número de cada carta
- Desarrollo de la mayor parte de la interfaz gráfica del juego.
- Implementación y ajuste de formularios de Windows Forms para la pantalla de juego.
- Mejoras de estilo visual en botones, fondos, tipografías y distribución de cartas rivales.

Nahima Suzette Rivera Hernández
Lógica del juego, mostrarImagenes.cs, ajustes visuales y reporte

- Programación del motor principal del juego en Juego.cs.
- Implementación del archivo mostrarImagenes.cs y del cargador de imágenes de cartas y dorsos.
- Conversión e integración de la paleta pastel para los recursos visuales.
- Ajustes visuales de la interfaz gráfica y elaboración del reporte del proyecto.

Karla Andrea Durán Segura
Base de datos, API y registro del log

- Creación y mantenimiento de la base de datos MySQL.
- Implementación de BaseDatos.cs y migración posterior de la comunicación a una API.
- Desarrollo de la API en Python y configuración de sus requerimientos.
- Implementación del registro de movimientos (log) y apoyo en la integración del repositorio.

Pedro Axel Llarena Padilla
Pantallas inicial y final de la interfaz gráfica

- Configuración inicial del proyecto en Windows Forms.
- Desarrollo de la pantalla inicial de la interfaz gráfica.
- Implementación de componentes visuales de entrada, selección de jugadores y navegación.
- Desarrollo de la pantalla final con el ganador y la información de cierre de la partida.
- Implementó FormFinal con podio, historial de partidas ganadas y perdidas, efectos de confeti y apertura automática al terminar la partida.

3. Actividades realizadas

Nahima Suzette Rivera Hernández
Lógica del juego, mostrarImagenes.cs, ajustes visuales y documentación

- Documentó las reglas oficiales de UNO utilizadas como base del proyecto. (5, 9)
- Programó el motor principal del juego en Juego.cs (implementó creación del mazo, barajado, reparto inicial, control de turnos y validación de jugadas, programación de efectos de Salta, Reversa, +2, Comodín y +4, así como la selección de color, declaración de UNO y conteo de puntos) . (11, 12)
- Preparó e integró la versión pastel de las cartas e implementó el cargador actual de imágenes y dorsos. (39, 40, 51)
- Realizó ajustes visuales en la interfaz de juego. (67)
- Se encargó de ir reuniendo la documentación del proyecto. (49, 68)

Selyan Nieto Aguirre
Interfaz gráfica del juego y Carta.cs

- Programó la clase Carta para representar color, tipo y número de cada carta. (6, 14, 17, 18)
- Desarrolló la pantalla principal de juego y su estructura base en Windows Forms. (52)
- Integró cartas de rivales, fondo pastel, tipografía Fredoka y ajustes de legibilidad en los botones. (53, 54, 55, 56, 57, 58)
- Agregó resumen del funcionamiento de la pantalla de juego y realizó limpieza de clases que ya no se utilizaban. (45, 46, 62)

Karla Andrea Durán Segura
Base de datos, API y log

- Creó el script de base de datos y las tablas necesarias para jugadores, partidas y movimientos. (7)
- Desarrolló BaseDatos.cs y corrigió columnas para mantener compatibilidad entre el código y la base de datos. (10, 13, 23, 24, 25, 26)
- Desarrolló la API en Python para comunicar el programa con la base de datos. (36, 38)
- Implementó la clase para registrar el log de movimientos mediante los eventos del juego e integró cambios a main. (60, 61)

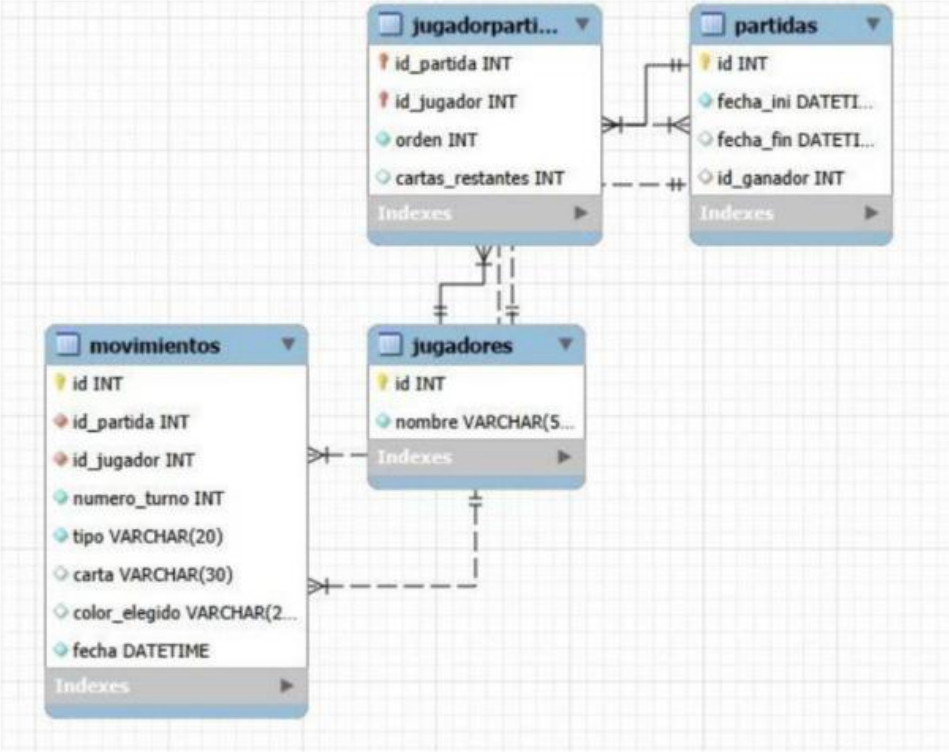
Pedro Axel Llarena Padilla
Interfaz de inicio y cierre de partida

- Configuró el inicio del proyecto de Windows Forms y el tamaño de la ventana. (20)
- Desarrolló la pantalla inicial con selección de jugadores, botones pastel, títulos y componentes visuales. (21, 22, 29, 30, 31, 32, 33, 34, 37)
- Integró consulta de jugadores y récord desde la base de datos, así como mensajes de reintento ante errores de conexión. (35, 47, 48)
- Participó en la integración de la interfaz final y en ajustes de rutas para el uso de imágenes pastel (implementó FormFinal con podio, historial de partidas ganadas y perdidas, efectos de confeti y apertura automática al terminar la partida). (43, 64, 65, 66, 69)

Al momento de este reporte se registran 70 commits en el repositorio, desde el 21 de septiembre hasta el 8 de octubre de 2026, con participación de todos los integrantes del equipo. El historial incluye la incorporación del reporte final y la integración de la pantalla final del juego.

4. Diagrama entidad-relación de la base de datos

La base de datos del proyecto relaciona la información de los jugadores con las partidas, la participación de cada jugador y los movimientos generados durante el juego.



5. Listado de commits

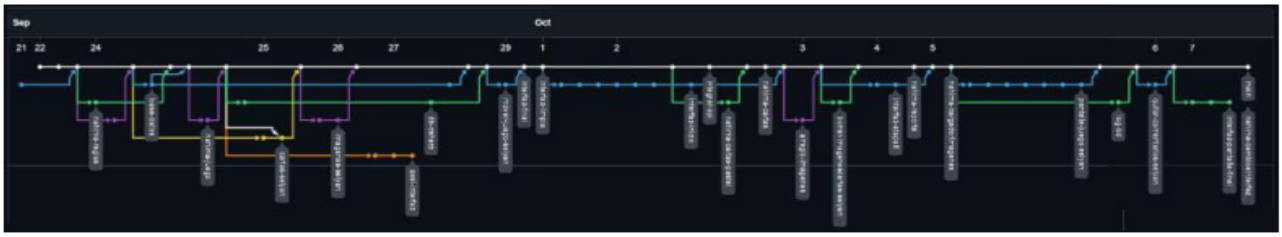
A continuación se presenta el historial de commits registrado desde el 21 de septiembre de 2026 hasta el avance actual. El historial muestra participación de los integrantes del equipo en distintas partes del proyecto.

Fecha	No.	Autor	Commit	Descripción
2026-10-08	70	Axllarena	d9ba496	Merge pull request #11 from karla-duran-segura/pantalla-final-podio
2026-10-07	69	Axllarena	88bc4c6	Pantalla final con podio, historial y confeti
2026-10-07	68	Nahima	c2bd9fb	Agregar reporte final del proyecto UNO
2026-10-07	67	Nahima	6257bb2	Agregar ajustes visuales a la interfaz de juego
2026-10-07	66	Axllarena	a61fc13	Abrir FormFinal al terminar la partida
2026-10-07	65	Axllarena	2b723b6	Mostrar historial de ganadas y perdidas en FormFinal
2026-10-07	64	Axllarena	3bd9de0	Agregar FormFinal base con titulo, tamano y fondo
2026-10-06	63	selyyannieto	959c0eb	Merge pull request #10 from karla-duran-segura/quitar-comentarios-selyan para ponerle comentarios
2026-10-06	62	Selyan	8bca346	Agregar resumen del funcionamiento de la pantalla de juego
2026-10-05	61	Karla	cbe6b80	Merge de log-bd en main
2026-10-05	60	Karla	dc3b0fd	Se hace la clase donde se regsitra el log de movimientos en la Base de Datos mediante los eventos del juego
2026-10-05	59	selyyannieto	f8145bc	Merge pull request #9 from karla-duran-segura/pantalla-juego-selyan
2026-10-05	58	Selyan	8df9344	Cambiar el color en los botones de la pantalla de juego a negro para que se lea mejor en el juego
2026-10-05	57	Selyan	55ba29f	Agregar colores configurables a BotonPastel para que sean Verdes como al inicio
2026-10-05	56	Selyan	44d3a1c	Agregar fuente Fredoka a la pantalla de juego
2026-10-05	55	Selyan	02b3796	Cambiar color de fondo y de la letra en la pantalla de juego para que se vea mejor
2026-10-05	54	Selyan	99058c0	Fondo liso en la pantalla de juego
2026-10-05	53	Selyan	66b4e5c	Pantalla de juego con estilo pastel y cartas de los rivales
2026-10-05	52	Selyan	0d844f4	Agregar pantalla de juego version 1 para ver si funciona el boton de comenzar
2026-10-05	51	Nahima	3f0dcef	Agregar cargador de imágenes
2026-10-05	50	Axllarena	efecd5c	Merge pull request #8 from karla-duran-segura/interfaz-etapa8
2026-10-04	49	Nahima	3179d19	Agregar el avance del reporte a la fecha: 04-10-2026
2026-10-04	48	Axllarena	6037c80	interfaz: mostrar quien reparte y quien comienza, asi como cartas iniciales
2026-10-04	47	Axllarena	e766a57	Manejo de error de conexion con boton Reintentar
2026-10-03	46	selyyannieto	f9c550b	Borrar la clase ImagenesCartas ya que no se usa en el codigo en ningun momento
2026-10-03	45	Selyan	31c0db7	Eliminar clase ImagenesCartas que no se usa
2026-10-03	44	Axllarena	7b424df	Merge pull request #6 from karla-duran-segura/arreglo-imagenes
2026-10-03	43	Axllarena	9c1fda4	arreglo: apuntar el csproj a la carpeta Imagenes_pastel
2026-10-02	42	Axllarena	3582aaf	Merge interfaz-inicio en main
2026-10-02	41	Nahima	8f46777	Elimina carpeta extra
2026-10-02	40	Nahima	b02ba88	Merge branch 'nahima-cartas-pastel' Se suben las cartas convertidas a los colores pasteles elegidos: azul, morado, rosa y amarillo.
2026-10-02	39	Nahima	79d9c7d	Subir imágenes de las cartas en color pastel
2026-10-02	38	Karla	6507b4c	Se agregó una carpeta con la API (archivo de python) y un archivo de texto con los requerimientos (instalaciones necesarias en python)
2026-10-02	37	Axllarena	40c1308	Se modifico el Form1.cs para que permita seleccionar 4 jugadores para iniciar la partida
2026-10-02	36	Karla	687de06	Ahora BaseDatos.cs no accede directamente a la base sql, se ha implementado la API (llamadas)
2026-10-02	35	Axllarena	03d6fa3	mostrar jugadores y su record desde la base de datos
2026-10-02	34	Axllarena	ee92a5e	Se agregaron botones pastel (Salir y Comenzar partida)
2026-10-02	33	Axllarena	2ceaf41	Se agregaron logo de cartas, titulo y textos del lado izquierdo
2026-10-01	32	Axllarena	3eca6b2	Se agrego el panel de vidrioy encabezado de se jugadores
2026-10-01	31	Axllarena	0f81e01	interfaz: agregar tema pastel y fondo con degradado
2026-10-01	30	Axllarena	d1f07c0	Se configuro ventana de inicio, titulo, tamano y DPI
2026-10-01	29	Axllarena	e655c0f	Configurar inicio de Windows Forms
2026-09-29	28	selyyannieto	c13b1c3	Merge pull request #5 from karla-duran-segura/mover-juego-selyan
2026-09-29	27	Selyan	85b8334	Mover Juego.cs dentro del proyecto para que corra todo mejor
2026-09-27	26	Karla	9d54b52	Se agrega la clase BaseDatos que tiene conexión a MySQL

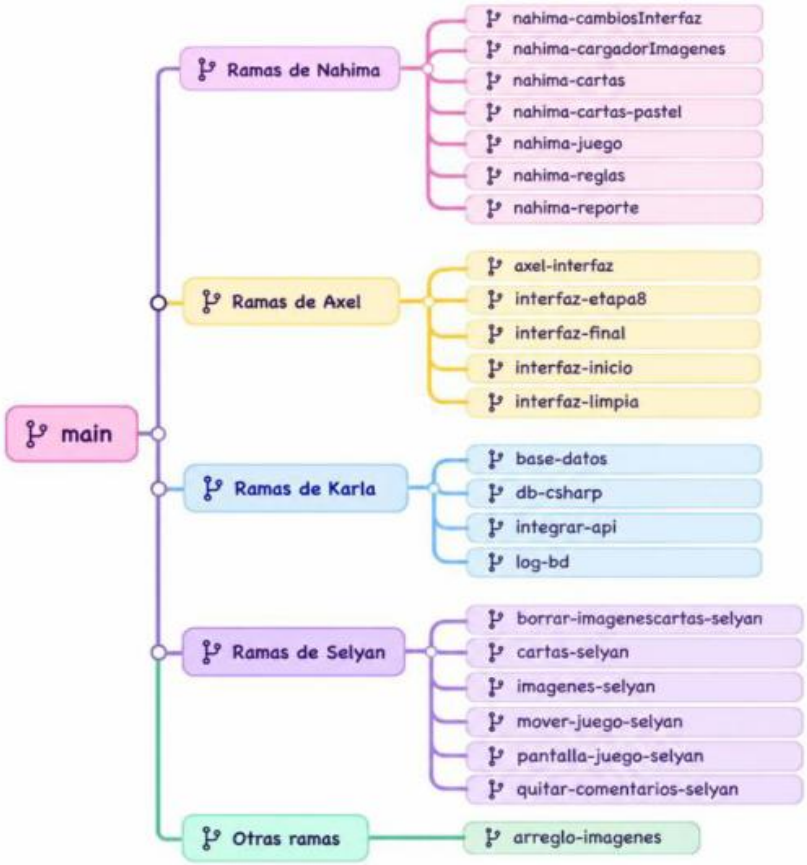
Fecha	No.	Autor	Commit	Descripción
2026-09-27	25	Karla	69a368d	Merge branch 'base-datos'
2026-09-27	24	Karla	0e17a60	Se corrigieron un par de detalles/columnas para que los nombres coincidieran con el nuevo archivo BaseDatos.cs
2026-09-27	23	Karla	c9d543b	Crear BaseDatos.cs con métodos de acceso a MySQL (uno es visual studio). También se incluye el archivo en uno.csproj
2026-09-27	22	Axllarena	c8904f7	Seleccionar listado de jugadores y boton de cotinuar con ellos
2026-09-27	21	Axllarena	83830e4	Agregar listado de jugadores y BaseDatos temporal en FormInicio
2026-09-26	20	Axllarena	a2402a8	Configurar proyecto como Windows Forms y crear FormInicio base
2026-09-26	19	selyyannieto	852c795	Juntar la rama de las imagenes con el main para que se pueda probar
2026-09-26	18	Selyan	cf6afb5	Se agregaron todas las imagenes de las cartas
2026-09-25	17	Selyan	4fb3d90	Agregar clase ImagenesCartas para cargar las imagenes de las cartas y no sea dificl manejarlas mas adelante
2026-09-25	16	selyyannieto	edce715	Merge pull request #3 from karla-duran-segura/cartas-selyan
2026-09-25	15	Selyan	267de4f	Merge branch 'main' of https://github.com/karla-duran-segura/UNO into cartas-selyan
2026-09-25	14	Selyan	96d15e0	Se agrego la clase Carta.cs al proyecto
2026-09-24	13	Karla	89e8ca6	Agregar paquete MySql.Data. Lo usaremos para conectar el programa de la base de datos al servidor local.
2026-09-24	12	Nahima	4d4f9c2	Integrar motor del juego UNO
2026-09-24	11	Nahima	cdd6864	Agregar motor del juego UNO
2026-09-24	10	Karla	4409986	Merge branch 'base-datos'
2026-09-24	9	Nahima	62f3342	Merge branch 'nahima-reglas' Commit reglas en las que basaremos toda la programación del juego
2026-09-24	8	Karla	b221d5c	Se agregó el script para la base de datos
2026-09-24	7	selyyannieto	a837889	Merge pull request #1 from karla-duran-segura/cartas-selyan
2026-09-24	6	Selyan	d950824	Se agrego una clase llamada Carta
2026-09-24	5	Nahima	8224c90	Documentar reglas y alcance de nuestro UNO
2026-09-22	4	Selyan	0512a8b	Merge branch 'main' of https://github.com/karla-duran-segura/UNO
2026-09-22	3	Selyan	b108d7e	Primer commit, creamos el proyecto
2026-09-22	2	Selyan	223e0c2	Add .gitattributes and .gitignore
2026-09-21	1	Karlita	c5b8a66	Initial commit

6. Listado de ramas generadas

Se presentan tres vistas complementarias del repositorio: el gráfico real de GitHub, el listado de ramas locales y remotas, y una agrupación ilustrativa por componentes.




```
* main
  nahima-cambiosInterfaz
  nahima-cargadorImagenes
  nahima-cartas
  nahima-cartas-pastel
  nahima-juego
  nahima-reglas
  nahima-reporte
  remotes/origin/HEAD -> origin/main
  remotes/origin/arreglo-imagenes
  remotes/origin/axel-interfaz
  remotes/origin/base-datos
  remotes/origin/borrar-imagenescartas-selyan
  remotes/origin/cartas-selyan
  remotes/origin/db-csharp
  remotes/origin/imagenes-selyan
  remotes/origin/integrar-api
  remotes/origin/interfaz-etapa8
  remotes/origin/interfaz-final
  remotes/origin/interfaz-inicio
  remotes/origin/interfaz-limpia
  remotes/origin/log-bd
  remotes/origin/main
  remotes/origin/mover-juego-selyan
  remotes/origin/nahima-cambiosInterfaz
  remotes/origin/nahima-cargadorImagenes
  remotes/origin/nahima-cartas
  remotes/origin/nahima-cartas-pastel
  remotes/origin/nahima-juego
  remotes/origin/nahima-reglas
  remotes/origin/nahima-reporte
  remotes/origin/pantalla-juego-selyan
  remotes/origin/quitar-comentarios-selyan
```



La visualización corresponde al gráfico de red de GitHub e ilustra las bifurcaciones y merges registrados, incluyendo el desarrollo de la pantalla final del juego.

7. Problemas encontrados y cómo se solucionaron

Problema	Situación encontrada	Solución aplicada
Integración de Juego.cs en el proyecto	El archivo del motor no se encontraba inicialmente en la ubicación más conveniente para compilarlo junto con el resto del proyecto.	Se movió Juego.cs dentro del proyecto y posteriormente se integró a main, lo que facilitó su compilación con los demás archivos.
Comunicación con la base de datos	La primera versión de BaseDatos.cs accedía directamente a MySQL, lo que complicaba la separación entre la aplicación y el almacenamiento.	Se desarrolló una API en Python y BaseDatos.cs se modificó para comunicarse mediante peticiones HTTP.
Diferencias de nombres entre código y base de datos	Algunos nombres de columnas no coincidían con los utilizados desde BaseDatos.cs.	Se corrigieron los nombres y detalles del esquema para mantener consistencia entre la base de datos y el código.
Manejo de imágenes de las cartas	El proyecto requiere muchas imágenes y la interfaz necesita acceder a ellas de forma uniforme, además de contar con un dorso para ocultar las manos rivales.	Se organizó el conjunto de recursos gráficos, se generó una versión pastel y se implementó un cargador centralizado para obtener la imagen de cada carta y su dorso.

Trabajo simultáneo con Git y GitHub	Cada integrante desarrolló partes distintas en ramas separadas, por lo que era necesario mantener el trabajo aislado sin perder cambios al integrarlo.	Se utilizaron ramas específicas, merges y pull requests para integrar los avances progresivamente a main.
Cambios visuales en la interfaz y cartas	Las imágenes y algunos elementos de la interfaz no correspondían por completo con la paleta pastel y la presentación final deseada.	Se crearon ramas de ajustes visuales para convertir cartas, mejorar tipografías, redistribuir elementos y hacer más legible la interfaz de juego.

8. Integración del Proyecto

El proyecto se encuentra organizado alrededor de cuatro componentes principales: motor del juego, recursos gráficos, interfaz y persistencia de datos. El motor contiene las reglas principales de UNO; la base de datos y la API cuentan con una estructura funcional; y los recursos visuales incluyen una versión pastel de las cartas.

La interfaz gráfica cuenta con una pantalla inicial desarrollada en Windows Forms, una pantalla principal de juego con estilo pastel y cartas de rivales, y componentes de cierre de partida. Asimismo, el sistema ya dispone de clases para la representación de cartas, el manejo del log de movimientos y el cargador de imágenes.

En términos de control de versiones, el repositorio registra 70 commits documentados en este reporte y múltiples ramas locales y remotas que respaldan el trabajo colaborativo del equipo. Entre los cambios más recientes se encuentran la pantalla final con podio, historial y confeti, así como la entrega del reporte final.

9. Funcionamiento de la API y comunicación con C# y MySQL

La persistencia de datos del proyecto se realiza mediante una API desarrollada en Python (archivo api_uno.py), que funciona como intermediaria entre la aplicación en C# y la base de datos MySQL.

El programa nunca se conecta directamente a MySQL: la clase BaseDatos.cs envía peticiones HTTP con datos en formato JSON a la API, la API ejecuta las consultas SQL y devuelve su respuesta, también en JSON.

C# (BaseDatos.cs) --petición HTTP + JSON--> API (FastAPI) --consulta SQL--> MySQL
C# (BaseDatos.cs) <--respuesta JSON-------- API (FastAPI) <--filas--------- MySQL

Esta separación aporta seguridad (el usuario y la contraseña de la base de datos solo existen en la API y no dentro del programa que usan los jugadores), independencia entre la interfaz y el almacenamiento, y validación centralizada de los datos antes de guardarlos.

9.1 Requerimientos y ejecución de la API

Para poder ejecutar la API en Python es necesario instalar primero las dependencias indicadas en el archivo de requerimientos del proyecto (requirements.txt). Esto se hace una sola vez con el siguiente comando: pip install -r requerimientos.txt

Las librerías principales que se instalan son FastAPI (el framework de la API), Uvicorn (el servidor que la mantiene en ejecución) y PyMySQL (la conexión con MySQL); Pydantic se instala junto con FastAPI. Además, MySQL debe estar encendido y la base de datos creada con el script SQL del proyecto, ya que la API se conecta con el usuario uno_user a la base uno.

Una vez instaladas las dependencias, la API se inicia desde la carpeta donde se encuentra api_uno.py con: uvicorn api_uno:app --port 8000

La API queda escuchando en http://127.0.0.1:8000, que es la dirección que usa BaseDatos.cs. Debe estar en ejecución antes de abrir el juego; de lo contrario, la pantalla inicial muestra un mensaje de error con el botón Reintentar. FastAPI también genera una documentación interactiva en la ruta /docs, útil para probar las rutas desde el navegador.

9.2 Conexión de la API con MySQL

La API usa la librería PyMySQL. La función conectar() abre una conexión con la base de datos uno usando el usuario uno_user, y cada operación abre su propia conexión y la cierra al terminar (bloque finally).

```
def conectar():
    return pymysql.connect(
        host="localhost",
        user="uno_user",
        password="uno1234",
        database="uno",
        charset="utf8mb4"
    )
```

Para consultar datos se usa consulta_a_json, que ejecuta un SELECT y convierte cada fila en un diccionario {columna: valor}. La lista resultante es la que FastAPI transforma en JSON. Para guardar datos se usa ejecutar, que corre un INSERT o UPDATE, confirma los cambios con commit() y devuelve el identificador generado (lastrowid).

```
def consulta_a_json(sql, parametros=()):
    conexion = conectar()
    try:
        cursor = conexion.cursor()
        cursor.execute(sql, parametros)

        columnas = []
        for columna in cursor.description:
            columnas.append(columna[0])

        resultado = []
        for fila in cursor.fetchall():
            registro = {}
            for i in range(len(columnas)):
                registro[columnas[i]] = fila[i]
            resultado.append(registro)

        return resultado
    finally:
        conexion.close()


def ejecutar(sql, parametros=()):
    conexion = conectar()
    try:
        cursor = conexion.cursor()
        cursor.execute(sql, parametros)
        conexion.commit()
        return cursor.lastrowid
    finally:
        conexion.close()
```

Todas las consultas reciben los valores como parámetros (%s) en lugar de unirlos al texto SQL. PyMySQL se encarga de escapar los valores, lo que previene ataques de inyección SQL.

9.3 Rutas de la API

Las rutas de consulta usan el método GET; las de escritura, POST (crear) y PUT (actualizar).

GET Devuelve el id y el nombre de los jugadores registrados. /historial

GET Devuelve las partidas ganadas y perdidas de cada jugador de turno/movimientos

POST Guarda un movimiento (jugar, robar, pasar, etc.). /partidas/{id}/terminar

PUT Registra el ganador, la fecha de fin y las cartas restantes.

9.4 Transformación y transporte de la información

De MySQL a JSON (consulta). Cada ruta GET llama a consulta_a_json y devuelve la lista de diccionarios; FastAPI la convierte automáticamente en JSON y la envía como respuesta HTTP. Por ejemplo, la ruta de jugadores:

```python
@app.get("/jugadores")
def obtener_jugadores():
    return consulta_a_json("SELECT id, nombre FROM Jugadores ORDER BY id")
```

Ejemplo de la respuesta que recibe C#:

[{"id": 1, "nombre": "Selyan"}, {"id": 2, "nombre": "Axel"}]

De JSON a la base de datos (guardado). Para las rutas que reciben datos se definen modelos con Pydantic. El modelo describe los campos que debe traer el JSON; FastAPI convierte el cuerpo de la petición en un objeto de Python y valida sus tipos (si falta un campo o el tipo es incorrecto, responde con error 422). Los campos opcionales (carta y color_elegido) pueden venir nulos. Después, la ruta inserta los valores en MySQL con ejecutar:

```python
class NuevoMovimiento(BaseModel):
    id_partida: int
    id_jugador: int
    numero_turno: int
    tipo: str  # "jugar", "robar", "pasar", "uno", "castigo"...
    carta: Optional[str] = None  # ej: "Rojo 5", "Comodín +4"
    color_elegido: Optional[str] = None
```

```python
@app.post("/movimientos")
def registrar_movimiento(datos: NuevoMovimiento):
    id_movimiento = ejecutar(
        """INSERT INTO Movimientos
           (id_partida, id_jugador, numero_turno, tipo, carta, color_elegido, fecha)
           VALUES (%s, %s, %s, %s, %s, %s, NOW())""",
        (datos.id_partida, datos.id_jugador, datos.numero_turno,
         datos.tipo, datos.carta, datos.color_elegido)
    )
    return {"id_movimiento": id_movimiento}
```

Ejemplo del JSON que envía C# a esta ruta:

{"id_partida": 1, "id_jugador": 2, "numero_turno": 5,

"tipo": "JUGAR", "carta": "Rosa 7", "color_elegido": null}

9.5 Comunicación desde C# (BaseDatos.cs)

En C#, la clase BaseDatos es la única que se comunica con la API. Usa un solo HttpClient compartido por todo el programa, configurado con la dirección de la API y un tiempo máximo de espera de 10 segundos:

```
private const string UrlBase = "http://127.0.0.1:8000";


private static readonly HttpClient http = new HttpClient
{
    BaseAddress = new Uri(UrlBase),
    Timeout = TimeSpan.FromSeconds(10)
};
```

Tres métodos auxiliares concentran el envío y la recepción de datos. Get hace peticiones de consulta; Enviar convierte un objeto de C# en JSON con JsonConvert.SerializeObject y lo manda con POST o PUT; y Leer obtiene el texto de la respuesta y lanza una excepción si la API respondió con un error. La instrucción .GetAwaiter().GetResult() hace que la llamada asíncrona espere su resultado, de modo que el resto del código la use como una llamada normal.

```
private static string Leer(HttpResponseMessage r)
{
    string texto = r.Content.ReadAsStringAsync().GetAwaiter().GetResult();
    if (!r.IsSuccessStatusCode)
        throw new Exception("Error de la API (" + (int)r.StatusCode + "): " + texto);
    return texto;
}

5 referencias
private static string Get(string ruta)
{
    return Leer(http.GetAsync(ruta).GetAwaiter().GetResult());
}

3 referencias
private static string Enviar(HttpMethod metodo, string ruta, object cuerpo)
{
    var req = new HttpRequestMessage(metodo, ruta)
    {
        Content = new StringContent(JsonConvert.SerializeObject(cuerpo), Encoding.UTF8, "application/json")
    };
    return Leer(http.SendAsync(req).GetAwaiter().GetResult());
}
```

Recibir datos (JSON a objetos de C#). ObtenerJugadores pide la lista a la API, interpreta el texto JSON como un arreglo (JArray) y construye un objeto JugadorBD por cada elemento, leyendo cada campo por su nombre:

```
public List<JugadorBD> ObtenerJugadores()
{
    var lista = new List<JugadorBD>();
    foreach (JObject fila in JArray.Parse(Get("/jugadores")))
        lista.Add(new JugadorBD { id = (int)fila["id"], nombre = (string)fila["nombre"] });
    return lista;
}
```

Enviar datos (objetos de C# a JSON). CrearPartida manda un objeto con la lista de ids de los jugadores, en el orden de los turnos. Los nombres de las propiedades del objeto (jugadores) son los

nombres de los campos del JSON, por lo que deben coincidir con los del modelo de Pydantic. La respuesta trae el id de la partida creada:

```
public int CrearPartida(List<int> idsJugadores)
{
    string json = Enviar(HttpMethod.Post, "/partidas", new { jugadores = idsJugadores });
    int id_partida = (int)JObject.Parse(json)["id_partida"];
    turnos[id_partida] = 0;
    return id_partida;
}
```

RegistrarMovimiento funciona igual, pero además lleva en un diccionario (turnos) el número del último movimiento de cada partida, porque la API exige el campo numero_turno:

```
public void RegistrarMovimiento(int id_partida, int id_jugador, string tipo, string carta = null, string color_elegido = null)
{
    int turno;
    turnos.TryGetValue(id_partida, out turno);
    turno++;
    turnos[id_partida] = turno;

    Enviar(HttpMethod.Post, "/movimientos", new
    {
        id_partida = id_partida,
        id_jugador = id_jugador,
        numero_turno = turno,
        tipo = tipo,
        carta = carta,
        color_elegido = color_elegido
    });
}
```

9.6 Transacciones

Las rutas que modifican varias tablas a la vez (crear una partida y terminarla) funcionan como transacciones. Por ejemplo, al crear una partida se inserta primero la partida y después cada jugador en JugadorPartida. Si todo se completa se confirma con commit(); si ocurre un error se deshace todo con rollback(), de modo que nunca queda en la base de datos una partida incompleta. Esta ruta también valida que haya al menos 2 jugadores y, si no, responde con un error 400.

```
@app.post("/partidas")
def crear_partida(datos: NuevaPartida):
    if len(datos.jugadores) < 2:
        raise HTTPException(status_code=400, detail="Se necesitan al menos 2 jugadores")

    conexion = conectar()
    try:
        cursor = conexion.cursor()
        cursor.execute("INSERT INTO Partidas (fecha_ini) VALUES (NOW())")
        id_partida = cursor.lastrowid

        for i in range(len(datos.jugadores)):
            cursor.execute(
                "INSERT INTO JugadorPartida (id_partida, id_jugador, orden) VALUES (%s, %s, %s)",
                (id_partida, datos.jugadores[i], i + 1)
            )

        conexion.commit()
        return {"id_partida": id_partida}
    except Exception:
        conexion.rollback()
        raise
    finally:
        conexion.close()
```

9.7 Librerías utilizadas para JSON y comunicación

La conversión de datos entre C# y Python se apoya en las siguientes librerías:

C# Paquete de NuGet para trabajar con JSON. JsonConvert.SerializeObject convierte objetos de C# en texto **(Json.NET)**

JSON para enviarlos a la API. JObject y JArray (del espacio Newtonsoft.Json.Linq) interpretan el JSON recibido y permiten

leer sus campos por nombre, por ejemplo fila["id"]. JTokenType.Null sirve para detectar valores nulos, como fecha_fin o el ganador de una partida que no ha terminado. System.Net.Http C# Incluida en .NET. HttpClient, HttpRequestMessage y StringContent realizan las peticiones GET, POST y PUT a la API y

envían el cuerpo como texto con tipo application/json. FastAPI Python Define las rutas y convierte automáticamente a JSON las listas y

diccionarios que devuelven; también transforma las fechas de

MySQL en texto, que C# lee como DateTime. Pydantic Python BaseModel define la estructura esperada del JSON recibido, lo

convierte en un objeto de Python y valida sus tipos de dato. PyMySQL Python Conecta Python con MySQL y ejecuta las consultas SQL con

parámetros %s.

9.8 Manejo de errores

Si la API recibe datos inválidos responde con un código de error (422 por formato incorrecto, 400 por menos de 2 jugadores) y, si falla una operación de escritura, se hace rollback. En C#, el método Leer convierte cualquier respuesta con error en una excepción con el código y el detalle. Las pantallas capturan esa excepción: la pantalla inicial muestra el mensaje con el botón Reintentar, y un fallo al guardar un movimiento no detiene la partida.

10. Interfaces del juego

Interfaz de Inicio:

Para iniciar el juego se deben seleccionar los 4 jugadores y dar clic en el botón de “Comenzar partida”, antes de esto se debe de haber iniciado la API.



Interfaz de Juego:



Interfaz de Fin de la Partida:

Se muestra al ganador de la partida (el jugador que juntó 500 puntos a lo largo de las rondas), y se colocan en un podio. En la zona de abajo se muestra el historial de las partidas ganadas y perdidas de cada jugador y luego un botón para volver a la interfaz de Inicio.



ENTREGA PARCIAL TRES

Proyecto: Desarrollo del juego UNO - Modalidad multijugador

11. Nueva etapa: juego entre computadoras

Durante el tercer parcial se amplió la aplicación UNO desarrollada con C# y Windows Forms, con el propósito de permitir partidas desde diferentes computadoras. Se conservaron las reglas del juego, la interfaz de colores pastel y el registro de información mediante una API en Python y una base de datos MySQL. La ampliación incluye una arquitectura cliente-servidor, clases para conexión TCP, estructuras para compartir el estado de la partida, pantallas de conexión y registro de jugadores.

El desarrollo incorporó modificaciones de FormJuego, FormRed, Form1, Juego.cs, Conexion.cs, EstadoRed.cs, BaseDatos.cs, el proyecto uno.csproj y la API. Los mensajes y los estados deben permitir coordinar acciones entre anfitrión y clientes, mientras que las pruebas de integración comprueban la comunicación entre máquinas.

12. División de tareas entre los integrantes

Pedro Axel Llarena Padilla | Registro de nuevos jugadores

FormRegistro.cs: formulario con campo para nombre, botones Registrar y Volver; permite registrar varios jugadores seguidos.

Form1.cs (FormInicio): botón “+ Registrar jugador” y recarga de la lista al regresar sin perder la selección previa. Se contempla Enter para registrar y Esc para cerrar.

Selyan Nieto Aguirre | Modelos y datos que circulan por la red

EstadoRed.cs: definición de EstadoRed (representación de la partida), JugadorRed, CartaRed y AccionRed, que expresa las acciones solicitadas por un cliente. Estos modelos organizan la información que deberá sincronizarse entre computadoras.

Nahima Suzette Rivera Hernández | Comunicación TCP entre computadoras

Conexion.cs: implementación de Mensaje con tipo y datos en JSON, así como los métodos Enviar(), Iniciar() y Cerrar() y los eventos MensajeRecibido y Desconectado. La clase utiliza TcpClient y puede ser usada por el anfitrión y por los clientes.

Karla Andrea Durán Segura | Pantalla de conexión e integración restante

FormRed.cs: pantalla previa a la partida. El anfitrión configura de 2 a 4 computadoras, muestra su IP y espera conexiones; los clientes ingresan la IP del anfitrión para conectarse. La siguiente etapa contempla abrir FormJuego e integrar los componentes restantes.

13. Flujo de conexión y partida

1. Registro: un jugador nuevo se guarda desde FormRegistro y aparece en la lista inicial.

2. Preparación: se abre FormRed; una computadora actúa como anfitriona y las demás como clientes.

3. Comunicación: Conexion.cs envía y recibe mensajes estructurados, mientras EstadoRed.cs representa la información compartida.

4. Partida: tras establecer las conexiones, las computadoras deberán abrir FormJuego y mantener sincronizados turnos,cartas y accion.



14. División de actividades del parcial tres

Los números entre paréntesis remiten al número correlativo de la tabla de commits del apartado 15. Se distinguen las tareas inicialmente previstas de los cambios atribuidos a cada autor en el historial

Integrante	Actividades acreditadas por commits
Nahima Suzette Rivera Hernández	Implementó la clase Conexion para comunicación TCP (76). Modificó la interfaz de inicio e integró su rama (93, 94). Ajustó Juego.cs y la ejecución conjunta de cliente y servidor (103). Actualizó la documentación y resolvió un conflicto de FormJuego (72, 73).
Pedro Axel Llarena Padilla	Agregó FormRegistro para registrar jugadores y su integración mediante pull request (79, 80). Actualizó Form1 con funcionalidades posteriores (102).
Selyan Nieto Aguirre	Incorporó FormRed y Conexion al proyecto; ajustó configuración de compilación (83, 84). Modificó FormJuego y FormRed (91), actualizó FormRed (99), y FormJuego / Juego.cs (100). Integró una versión de partida en red con servidor y cuatro clientes (101). Ajustó el puntaje para ganar a 50 durante pruebas (105).
Karla Andrea Durán Segura	Adaptó BaseDatos.cs y la API para el modelo cliente-servidor (75). Agregó EstadoRed.cs y fusionó esa rama (77, 78). Modificó Form1 y su capacidad para conectar cuatro computadoras (81, 82, 89, 90). Ajustó IP del servidor, registro y BaseDatos (95–98); realizó pruebas y ajustes de IP (104, 106).

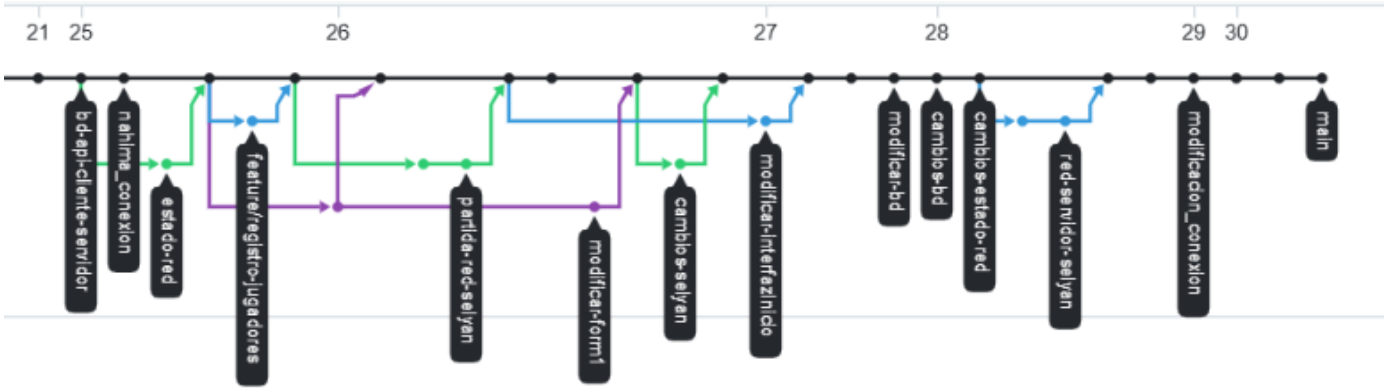
15. Listado de commits posteriores al reporte anterior

Continuación cronológica del registro anterior: el reporte parcial dos documentó los commits 1–70.

N.º	Fecha	Autor	Commit	Descripción registrada
71	2026-10-08	Karla	c2e4db9	Cambio del color del fondo del juego
72	2026-10-11	Nahima	2ef5f97	Actualizar reporte final del proyecto
73	2026-10-11	Nahima	30bc707	Resolver conflicto en FormJuego
74	2026-10-21	Karla	9d34d63	Limpieza de código en FormJuego
75	2026-10-25	Karla	1b94f93	Modificación en la clase basedatos.cs y en la api para que se pueda jugar usando el modelo cliente-servidor
76	2026-10-25	Nahima	e4ac317	Agregar clase Conexion para comunicación TCP entre jugadores
77	2026-10-25	Karla	86e73cb	Se agregó EstadoRed.cs para poder usarse en el modo red
78	2026-10-25	Karla	e9c9079	Merge branch 'estado-red'
79	2026-10-25	Axllarena	ef68bc5	Agregar formulario de registro de jugadores
80	2026-10-25	Axllarena	dd0c026	Merge pull request #12 from karla-duran-segura/feature/registro-jugadores
81	2026-10-26	Karla	b092f69	Se modificó form1 debido a la nueva forma de usarlo (cliente-servidor)
82	2026-10-26	Karla	28b82eb	Merge branch 'modificar-form1'
83	2026-10-26	Selyan	2f9806d	Agregar FormRed e incluir Conexion en el proyecto
84	2026-10-26	Selyan	d2d89ec	Agregar FormRed para partida en red y corregir plataforma Debug en csproj
85	2026-10-26	selyyannieto	8d5700a	Merge pull request #13 from karla-duran-segura/partida-red-selyan
86	2026-10-26	Nahima	041d2f7	index on main: e9c9079 Merge branch 'estado-red'
87	2026-10-26	Nahima	5adcc17	WIP on main: e9c9079 Merge branch 'estado-red'
88	2026-10-26	Karla	0bbb136	modificación en uno.csproj para corrección de error
89	2026-10-26	Karla	d5a1a1a	Modificación para la aceptación de 4 computadoras
90	2026-10-26	Karla	20b94fb	Merge branch 'modificar-form1'
91	2026-10-26	Selyan	9fd710e	Cambios en FormJuego y FormRed
92	2026-10-26	selyyannieto	af61f8f	Merge pull request #14 from karla-duran-segura/cambios-selyan
93	2026-10-27	Nahima	72b55cb	Modificación de la interfaz de inicio
94	2026-10-27	Nahima	bdae853	Merge branch 'modificar-interfazInicio' Modificar la interfaz de inicio.
95	2026-10-27	Karla	3c20481	Asignar la IP del servidor para guardar el registro de nuevos jugadores
96	2026-10-27	Karla	12451f5	Asignar ip de ethernet del server en este momento
97	2026-10-28	Karla	0296827	Cambios en la clase basedatos
98	2026-10-28	Karla	3a2b03e	Cambios es estadored para la correcta actualización y envios de estados
99	2026-10-28	Selyan	b570c12	Actualizar FormRed
100	2026-10-28	Selyan	9ea05c7	Actualizar FormJuego y agregar IndiceSinUno en Juego
101	2026-10-28	selyyannieto	0ecbaab	Partida en red con servidor y 4 clientes
102	2026-10-28	Axllarena	00892a3	Actualizar Form1 con nuevas funcionalidades

103	2026-10-29	Nahima	968ab76	Realizar modificación necesario para correr correctamente clliente y servidor, por consiguiente, cambios en Juego.cs
104	2026-10-30	Karla	e285770	cambios para prueba de conexión
105	2026-10-30	Selyan	ca877eb	Bajar puntos para ganar la partida a 50
106	2026-10-30	Karla	3447f96	ip prueba

16. Ramas generadas y utilizadas



Inventario completo de ramas:

Rama	Rama	Rama
arreglo-imagenes	axel-interfaz	base-datos
bd-api-cliente-servidor	borrar-imagenescartas-selyan	cambios-bd
cambios-estado-red	cambios-selyan	cartas-selyan
db-csharp	estado-red	feature/registro-jugadores
imagenes-selyan	integrar-api	interfaz-etapa8
interfaz-final	interfaz-inicio	interfaz-limpia
interfaz-pantalla-final	log-bd	main
modificacion_conexion	modificar-bd	modificar-form1
modificar-interfazInicio	mover-juego-selyan	nahima-cambiosInterfaz
nahima-cargadorImagenes	nahima-cartas	nahima-cartas-pastel
nahima-conexion	nahima-juego	nahima-reglas
nahima-reporte	nahima-reporteFinal	nahima_conexion
pantalla-final-podio	pantalla-juego-selyan	partida-red-selyan
quitar-comentarios-selyan	red-servidor-selyan	

17. Problemas encontrados y soluciones aplicadas

Problema	Situación detectada	Corrección documentada
Conflictos al integrar la interfaz	Los cambios desarrollados en distintas ramas podían entrar en conflicto al fusionarse.	Se resolvió un conflicto en FormJuego (73) y se integraron modificaciones con merges y pull requests (80, 85, 92, 94).
Preparación para cliente-servidor	La base de datos y la API de la etapa anterior se habían diseñado para una ejecución local.	Se modificaron BaseDatos.cs y la API para el modelo cliente-servidor (75); posteriormente se hicieron cambios adicionales a BaseDatos (97).
Conexión de varios equipos	El sistema debía identificar la máquina anfitriona, aceptar participantes y disponer de una IP alcanzable desde la red.	Se amplió la aceptación de cuatro computadoras (89), se ajustaron IP del servidor (95, 96) y se registraron pruebas de conexión/IP (104, 106).
Sincronización de la partida	Las computadoras requieren datos compartidos y actualización consistente de turnos y estados.	Se incorporó EstadoRed.cs (77) y se revisó su actualización y envío de estados (98); se ajustaron FormJuego y Juego.cs (100, 103).
Configuración del proyecto	El proyecto presentó necesidades de ajuste en plataforma Debug y archivo de proyecto.	Se corrigió la plataforma Debug en uno.csproj (84), seguida de una modificación adicional del archivo (88).

Ajustes del modo red	La interfaz y la lógica requirieron cambios durante las pruebas de integración.	Se actualizaron FormRed y FormJuego (91, 99, 100), la interfaz principal (93, 102) y el umbral de puntos para facilitar pruebas (105).

18. Actualización técnica: API, base de datos, TCP y JSON

18.1. Arquitectura.

La etapa anterior utilizaba una API Python (FastAPI) como intermediaria entre C# (BaseDatos.cs) y MySQL. En el parcial tres se registró una modificación de BaseDatos.cs y de la propia API para admitir el modelo cliente-servidor (75), junto con ajustes posteriores a la clase de acceso a datos (97). Por ello, la explicación de la API de las páginas 10–15 debe leerse como la implementación anterior a estos cambios.

18.2. Persistencia y configuración de servidor.

Los commits documentan la asignación de la IP del servidor para registrar jugadores (95), el uso de una IP Ethernet en pruebas (96) y nuevas pruebas de IP y conexión (104, 106). Esto indica que la aplicación ya no puede asumir en todas las computadoras el mismo localhost de la configuración original. La dirección y el puerto definitivos deben confirmarse en la versión actual del código; no se incorporan valores de red transitorios al reporte.

18.3. Mensajería.

Conexion.cs introduce comunicación TCP entre los jugadores (76). Según el diseño documentado en el parcial tres, los mensajes contienen tipo y datos estructurados en JSON, y la clase contempla operaciones de envío, inicio y cierre y eventos de recepción/desconexión. TCP transporta mensajes entre máquinas; JSON representa los datos intercambiados. La API HTTP/JSON que guarda datos en MySQL es una comunicación distinta de la conexión TCP usada para coordinar la partida.

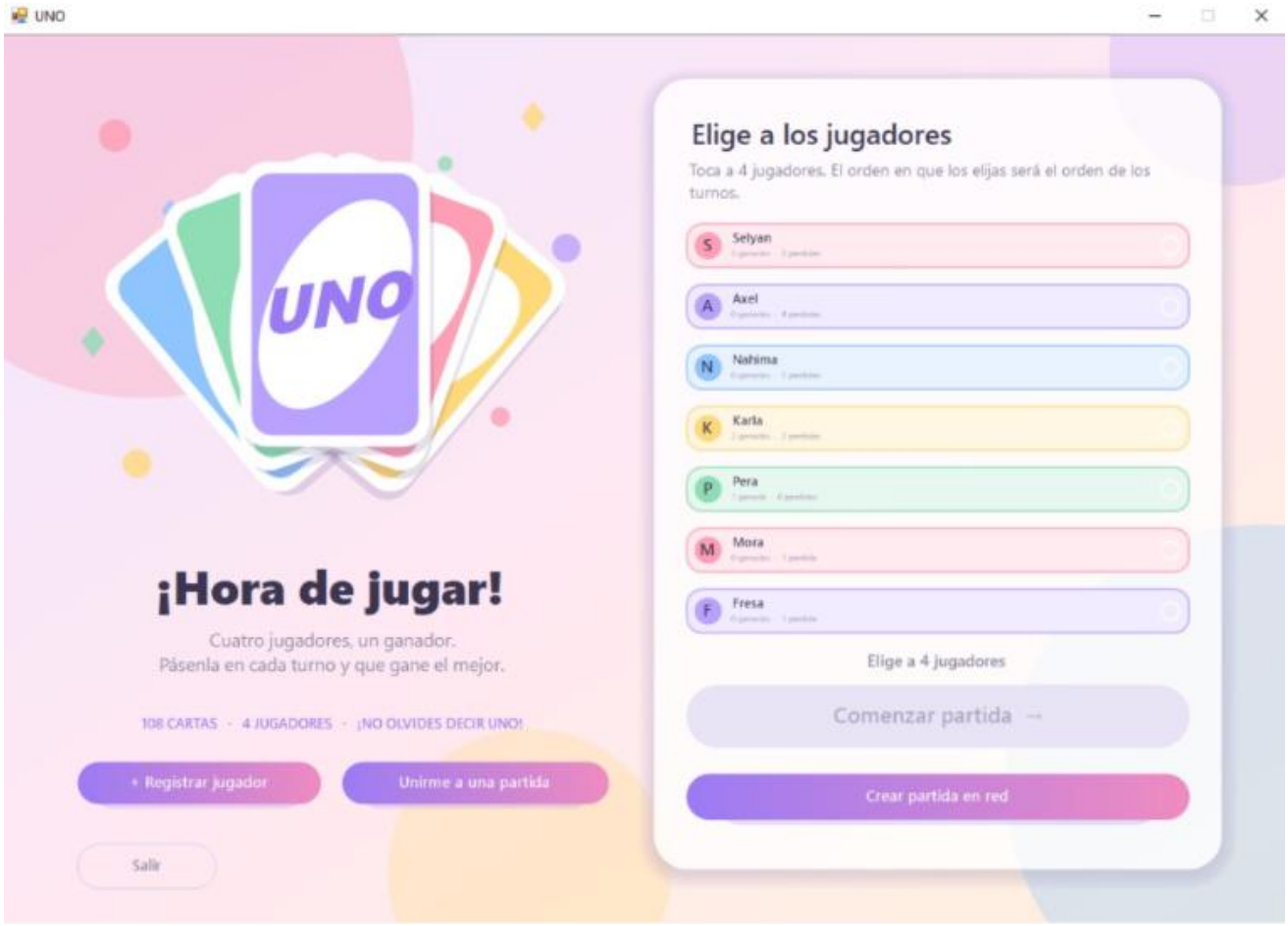
18.4. Estado y acciones.

EstadoRed.cs (77) reúne modelos para representar información de la partida; se modificó después para actualizar y enviar estados correctamente (98). Los ajustes a FormRed, FormJuego y Juego.cs (83, 84, 91, 99–101, 103) apoyan la integración entre la lógica local y la información compartida.

18.5. Alcance de la validación.

El historial demuestra incorporación de componentes, fusiones y pruebas, pero no ofrece por sí solo resultados reproducibles de todas las conexiones, partidas completas, desconexiones o consistencia de la base de datos. Se recomienda adjuntar evidencia de ejecución con cuatro clientes, registro de usuarios y finalización de una partida para acreditar esas condiciones.

19. Nuevas interfaces






UNO - Fin de la partida
Fin de la partida
Pera ganó la partida
Pera
Mora
Fresa
2
0 pts
1
173 pts
3
0 pts
4. Nahima 0 pts
Historial de partidas
Pera: 1 ganadas · 0 perdidas
Fresa: 0 ganadas · 1 perdidas
Mora: 0 ganadas · 1 perdidas
Nahima: 0 ganadas · 5 perdidas
Volver al inicio

20. Conclusión

El desarrollo de nuestro proyecto UNO representó mucho más que la creación de un videojuego. Fue una oportunidad para transformar conocimientos teóricos en soluciones reales, enfrentar desafíos técnicos y comprender que el éxito de un proyecto no depende únicamente de la calidad del código, sino también de la capacidad de quienes lo desarrollan para organizarse, colaborar y asumir responsabilidades.

Comprendimos que la autonomía no significa trabajar de forma aislada, sino tener la iniciativa y el compromiso necesarios para aportar al cumplimiento de un objetivo compartido.

Las incompatibilidades, los errores de integración y los desafíos de comunicación entre computadoras nos impulsaron a fortalecer nuestra capacidad de análisis, adaptación y resolución de problemas. Aprendimos que un equipo verdaderamente comprometido no es aquel que nunca enfrenta obstáculos, sino aquel que encuentra la manera de superarlos.

Este proyecto también nos permitió comprender que el trabajo en equipo requiere mucho más que dividir actividades: exige comunicación, confianza, responsabilidad y disposición para apoyar a los demás cuando las circunstancias lo requieren.

Finalmente, consideramos que el mayor resultado de este proyecto no se encuentra solamente en el producto desarrollado, sino en las habilidades, experiencias y aprendizajes que construimos durante el proceso. Nos llevamos una visión más completa de lo que implica desarrollar software y la convicción de que la organización, la autonomía y la colaboración son fundamentales para alcanzar objetivos cada vez más ambiciosos.

Porque al final, este proyecto nos demostró que los grandes resultados no se construyen únicamente con buenas ideas, sino con personas capaces de convertirlas en realidad. Y que cuando el compromiso individual se une a un propósito colectivo, el verdadero logro deja de pertenecer a una sola persona para convertirse en el éxito de todo un equipo.